



Enterprise Integration using Apache Camel

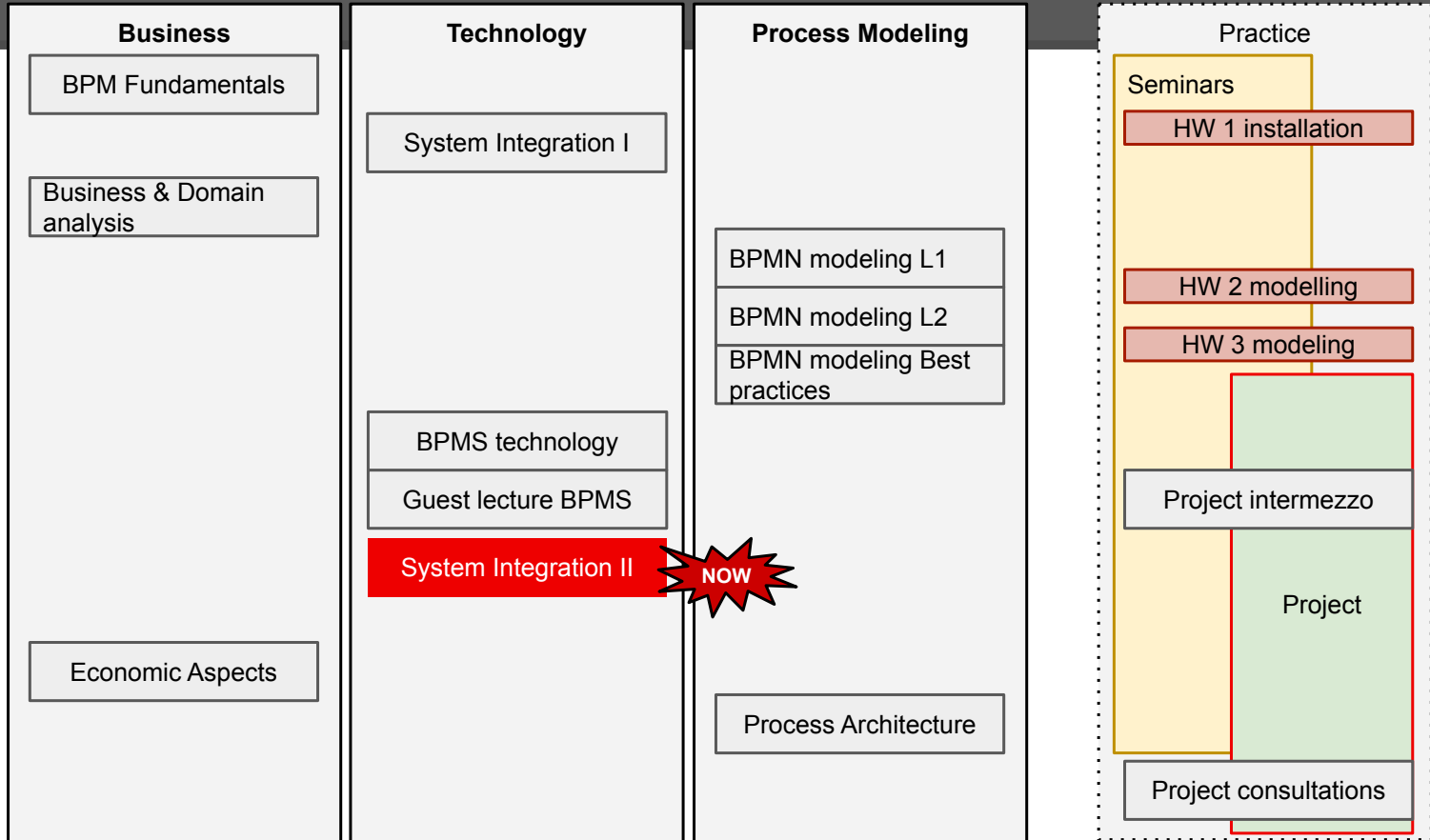
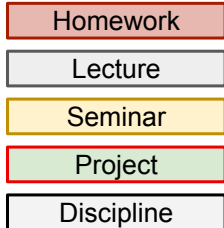
PV207 - Business Process
Management

Mgr. Ivo Bek
Principal Product Manager

PV207 helicopter view

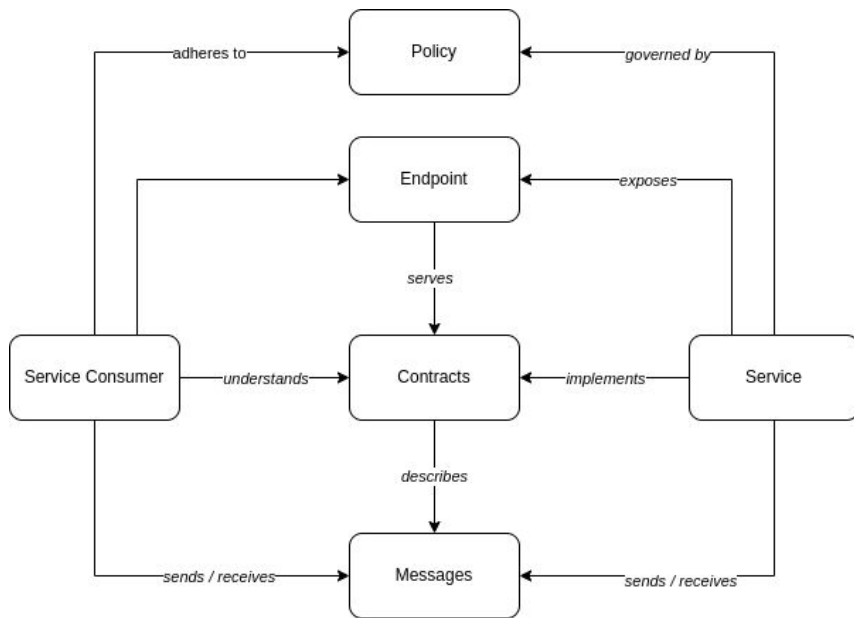
Semester time

Legend:



Introduction to Enterprise Integration

Recap

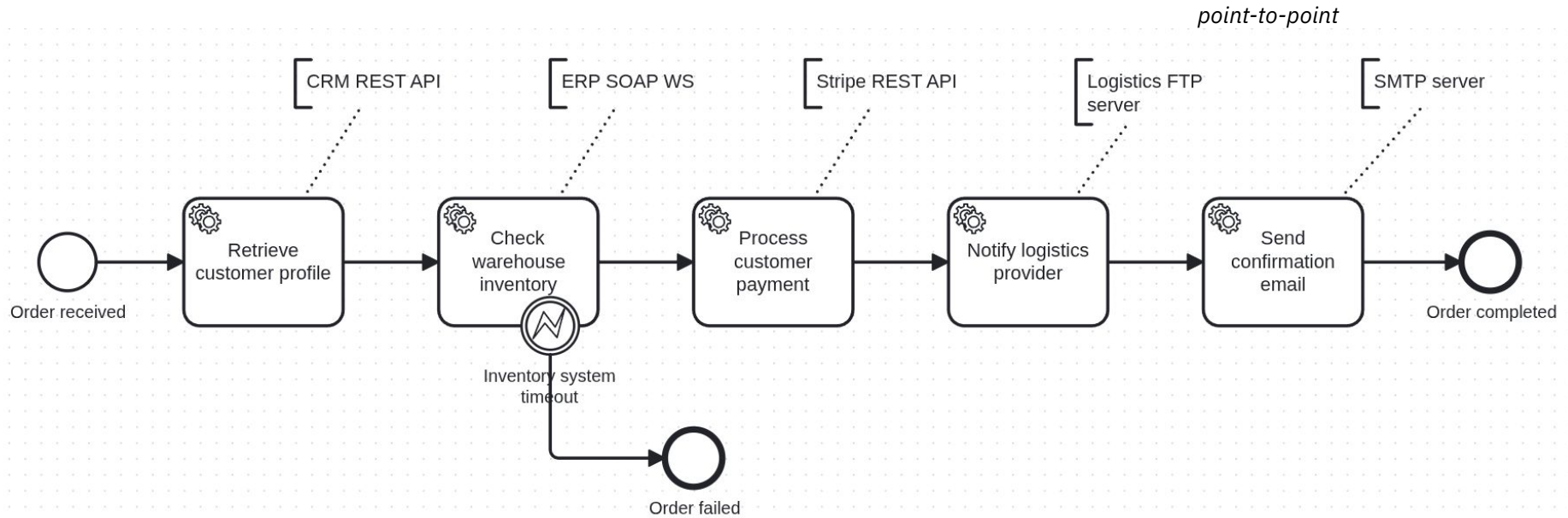


- ▶ **Interface characteristics**
data formats, transmission protocols, data objects, interaction types, integrity (validation, transactional, stateful, idempotence), security, reliability, error handling, performance
- ▶ **Communication and API architectural styles**
REST/OpenAPI, SOAP WS, gRPC, Message-driven, Streaming/Event-driven, ...
- ▶ **Enterprise Integration Patterns**
routing, publish-subscribe, correlation, content enricher, aggregation, retry, error handling, data transformation, splitter, message filter

What we'll discuss today

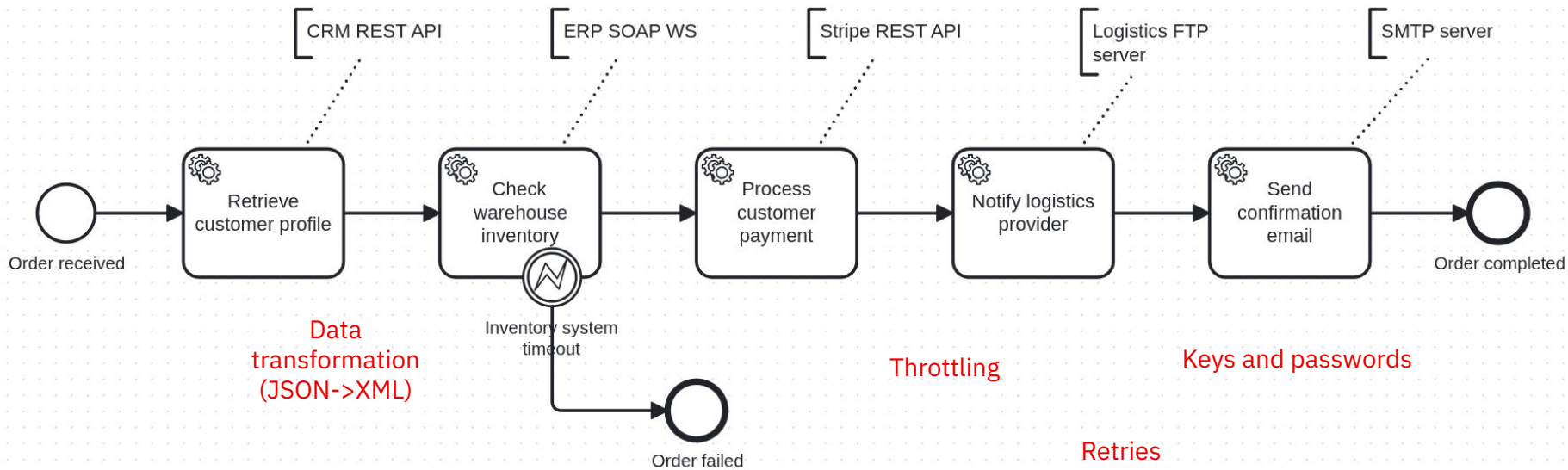
- ▶ Orchestration vs Integration
- ▶ Apache Camel
- ▶ Camunda + Camel
- ▶ Camel integration language
- ▶ Examples

BPMN process is the plan



Imagine managing 100s of business process definitions over decades

Integrations fail in the space between boxes



Data transformation
(JSON->XML)

Throttling

Keys and passwords

Retries

Service location changes
<multi/hybrid-cloud>

Network instability

API changes

Hidden assumptions

4xx errors
5xx errors

TLS / secure communication -
certificates

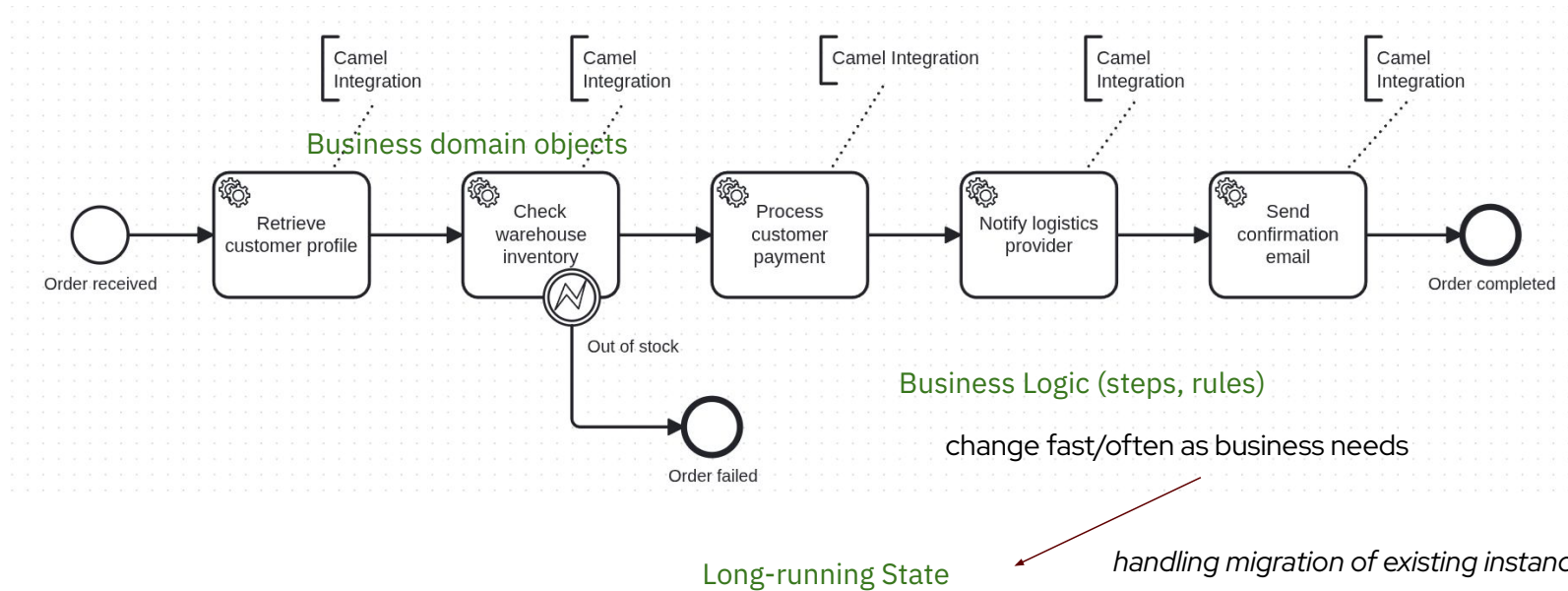
Separate **Orchestration** from **Integration**



Camel Integration acts as translator between the business process and the outside world

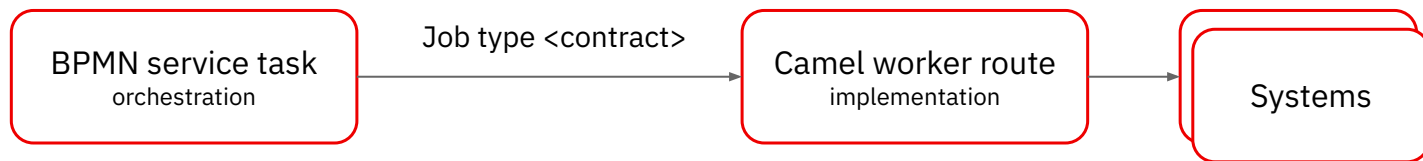
Data Transport

Data Transformation



Camunda orchestrates | Camel integrates

Camel makes the hidden assumptions explicit



Camunda: “What happens next?”

Camel: “How data moves and changes?”

- Contracts (OpenAPI)
- Transformation + validation
- Configurable endpoints
- Deliberate error semantics



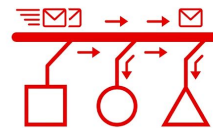
Replaces thousands of lines of custom code with pre-built, heavily tested **integration components**



Provides **built-in resilience** for error handling, automatic retries, and data formatting to prevent silent failures and lost data



This is
Apache Camel



Implements **Enterprise integration patterns** ensuring all data flows are built predictably rather than relying on one developer's unique coding style

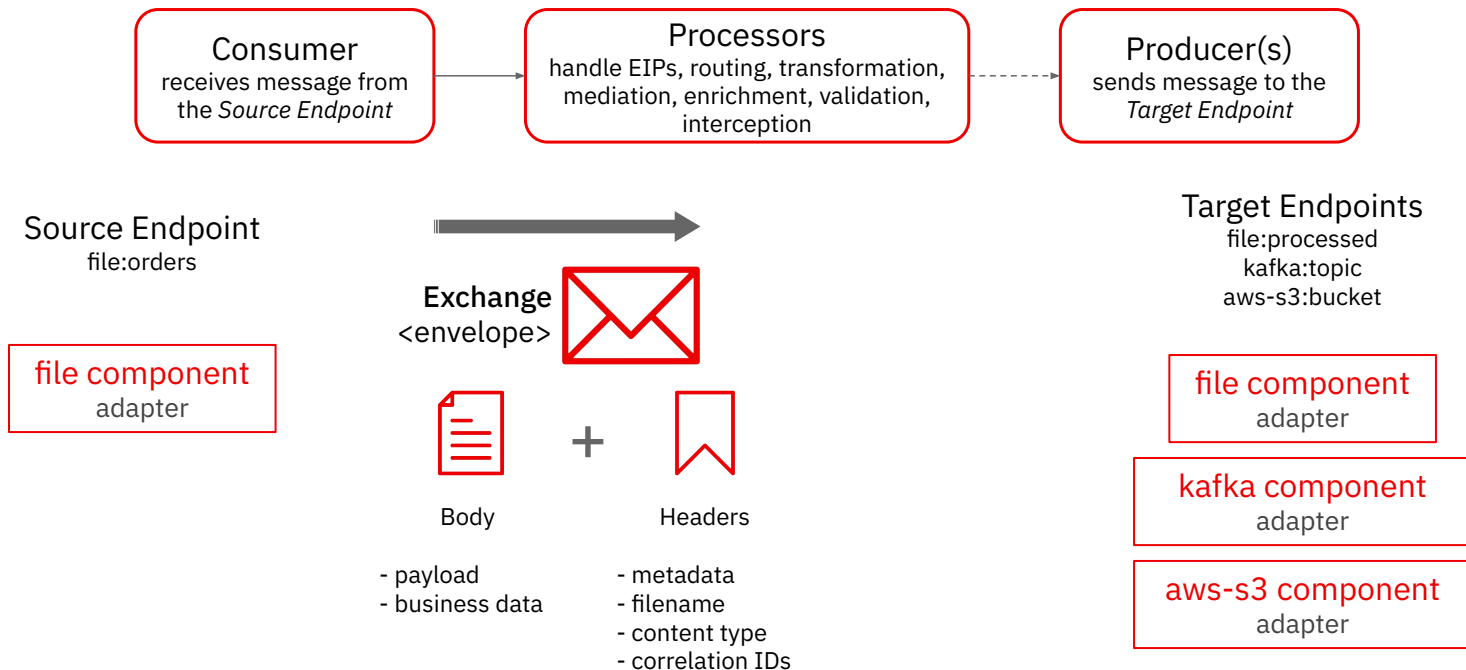
```
from("kafka:topic")  
  .to("grpc:endpoint")
```

Defines easy-to-read **integration logic** in XML, YAML or Java ensuring industry-standards

Camel Route

a message path over an Exchange

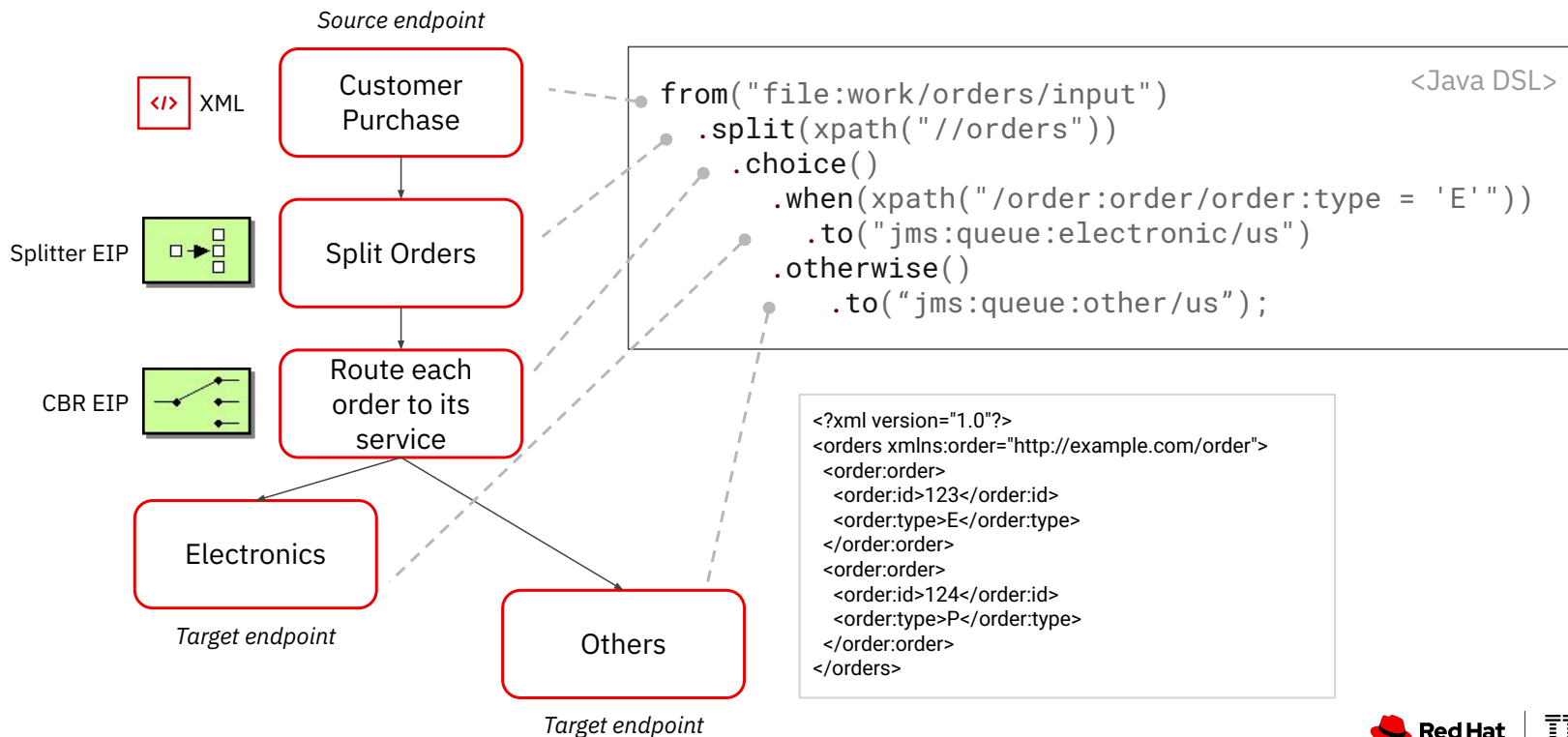
Camel route dictates how a message is consumed from a source endpoint, processed or transformed and delivered



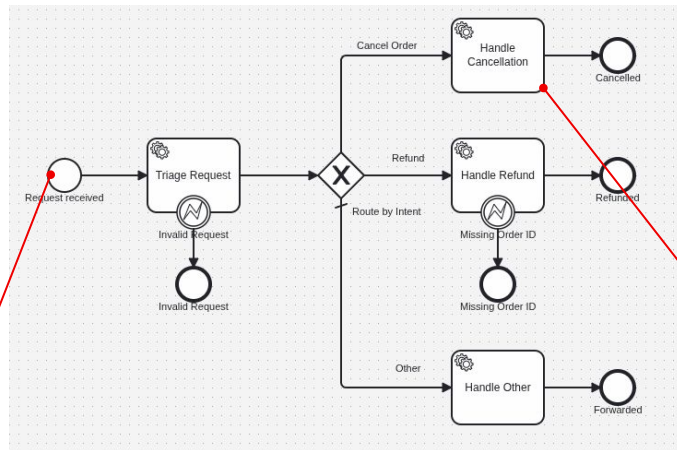
Every route has the same skeleton

```
- route:
  from:
    Source —• uri: timer
      parameters:
        period: "1000"
        timerName: hello
      steps:
        Processor —• - setBody:
          simple:
            expression: Hello World!
        Target —• - log:
          message: ${body}
```

If you can read Camel Language, you can edit real integrations



Camel routes are the integration layer around Camunda



Who triggers?
What's the contract in/out?
What semantics on failure?

Start processes (producer)

`to("camunda://startProcess")`

camunda component
adapter

Implement workers (consumer)

`from("camunda:worker?jobType=handle-cancel")`

camunda component
adapter

Enterprise Integration Patterns (moves) explain intent

how you shape & control flow

Content-based Router



```
.choice()  
.when(<expression>  
  <steps>  
.otherwise()  
  <steps>
```

Filter



```
.filter(<expression>  
  <step(s)>  
.end() -- optional
```

Message Translator



```
.setBody(<expression>  
.transform(<expression>  
.bean(<customBean>  
.to() -- xslt, jslt, ..
```

Splitter



```
.split(<expression>
```

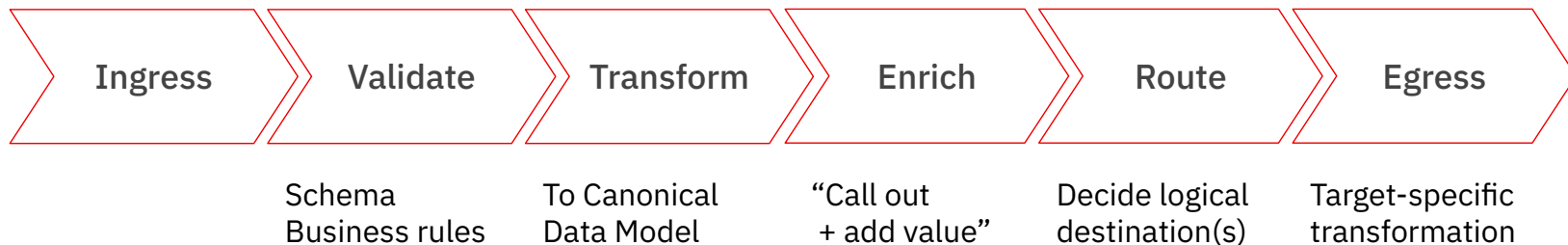
And more at

<https://camel.apache.org/components/latest/eips/enterprise-integration-patterns.html>

<https://www.enterpriseintegrationpatterns.com/patterns/messaging/>

Most integrations follow one recipe

API endpoint
Event listener
Polling file adapter



Data-centric | Decoupled

Most integration work is translation, not transport

unmarshal → transform → marshal

- unmarshal:
 - json: {}
- transform:
 - simple:
 - expression: '{ "customerId": "\${body[id]}", "email": "\${body[email]}" }'
- marshal:
 - json: {}

Business errors should take BPMN path

```
- validate:  
  expression:  
    simple: ${body} != null && ${header.correlationId} != null
```

Invalid state/data = business error (handled outcome)

```
- routeConfiguration:  
  id: business-error-handling  
  onException:  
    - onException:  
      exception:  
        - org.apache.camel.support.processor.PredicateValidationException  
      handled:  
        constant: "true"  
      steps:  
        - log:  
          message: "Validation failed, throwing BPMN error..."  
          loggingLevel: WARN  
        - throwException:  
          description: "Throw BPMN error"  
          exceptionType: "BusinessBpmnError"  
          message: "INVALID_REQUEST: Customer message and correlationId are required"
```

BPMN error

Boundary error

Model business outcomes in BPMN paths

Technical failures should retry

Timeout

Error 5xx

Connection reset

= technical failure

RETRY / BACKOFF

INCIDENT

```
- onException:
  exception:
    - java.net.ConnectException
    - java.net.SocketTimeoutException
  redeliveryPolicy:
    maximumRedeliveries: 3
    redeliveryDelay: 5000
  steps:
    - log:
        message: "Upstream technical failure
(retrying): ${exception.message}"
```

Retry things that might
succeed later

Portable Integrations

using externalized configurations

- ▶ Same route, different env
- ▶ Put in properties: URLs, timeouts, retries, toggles

application.properties

```
crm.host = {{env:CRM_HOST:crm.example.com}}  
camel.component.http.connectTimeout = {{configmap:crm/connectTimeout:2000}}
```

route.camel.yaml

```
- to:  
  uri: "https://{{crm.host}}/api/customers"
```

Make applications externally configurable

and define placeholders instead of the actual values

in properties file, environment variables, configmaps, secrets, etc.

Placeholder functions
env, sys (jvm properties), bean, service

Kubernetes placeholder functions
configmap, secret

Autoconfiguration

camel.component.name.option1=value1
camel.kamelet.name.option2=value2
camel.dataformat.name.option3=value3



```
db.host = 127.0.0.1
db.port = 8080
db.user = ibek
db.pwd = {{secret:db/key}}
file.path = file.json
camel.component.openai.apiKey={{env:OPENAI_API_KEY}}
```

application.properties

```
.to("file:foo?fileName={{file.path:/some/path}}") -- with default value
.to("file:foo?bufferSize={{?myBufferSize}}") -- optional
.log("What {{configmap:myconfig/drink}} do you want?")
.from("file:{{bean:foo.bar}}")
.to("{{env:HOME}}")
.to("{{sys:file.separator}}")
```

A method for reading and editing Camel routes

- ▶ Find entry/exit + contract boundary from, to
- ▶ Trace transformations + decisions setBody, jq, choice, when
- ▶ Locate config placeholders application.properties
- ▶ Follow error paths (technical vs business) onException
- ▶ Verify headers (routing/correlation) validate

Integration quality is shared across team roles

- ▶ **Business analyst:** required fields + business error semantics
- ▶ **Process analyst:** where routing/validation/enrichment belongs
- ▶ **Developer:** maintainable Camel YAML routes + clear error handling
- ▶ **Project manager:** Non-functional requirements as acceptance criteria (timeouts/retries/SLA)

BPMN + Camel



Business
Orchestration



Technical
Integration

BPMN designs the
process intent

Camel implements +
operationalizes reality

Camel Lab today

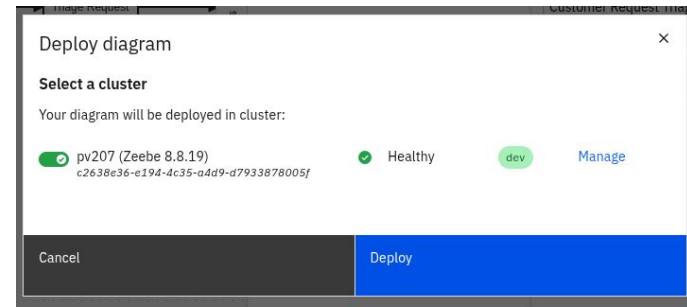
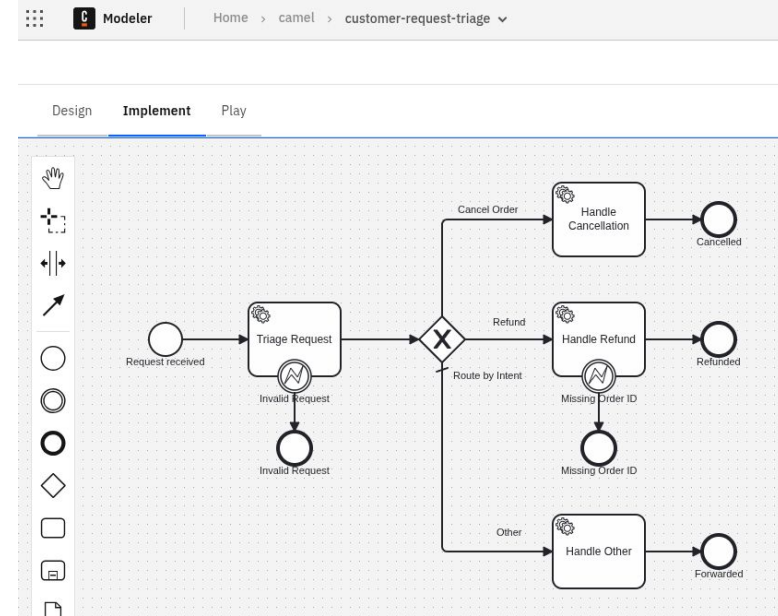
Customer Request Triage
Apache Camel + Camunda 8 SaaS

Camunda 8 SaaS

Prerequisite #1

- ▶ <https://accounts.camunda.io/signup>
- ▶ You should already have the access
- ▶ <https://modeler.camunda.io/>

- 1) Download
<https://raw.githubusercontent.com/ibek/pv207-camel-lab/refs/heads/main/customer-request-triage/customer-request-triage.bpmn>
- 2) Open <https://modeler.camunda.io/> and Create a new project “**camel**”
- 3) Upload **custom-request-triage.bpmn**
- 4) Open the model and **Deploy**



Camunda API Client

Create new client credentials

Client Name

camel

can contain letters, dashes, underscores, and digits

Scopes

Scopes define the access for client credentials.

Granular authorizations for the Orchestration Cluster API can be managed via Orchestration Cluster Identity.

Read more about [Scopes for Client Credentials](#).

☒ Orchestration Cluster API

☐ Optimize API

☐ Administration API - Resource: Secrets

Cancel

Create

Client credentials

Select a format for your credentials. For a full list of our clients, see [Working with APIs & tools](#).

 The client secret will not be shown again.

Spring Boot

Spring Boot YAML

Env Vars

Desktop Modeler

Export all credentials as environment variables to be used in a Linux environment to run an application written in any language.

```
export ZEEBE_ADDRESS='c2638e36-e194-4c35-a4d9-d7933878005f.bru-2.zeebe.camunda.io'
export ZEEBE_CLIENT_ID='H9rdT9~qVTQDKLyF0Pjj7WhK91bKzas'
export ZEEBE_CLIENT_SECRET='q33jL7WYRa~AMXp0nGjNBP6UmH1d0Seu_HFp0dDHEfXJT-jWSUsI9cf'
export ZEEBE_AUTHORIZATION_SERVER_URL='https://login.cloud.camunda.io/oauth/token'
export ZEEBE_REST_ADDRESS='https://bru-2.zeebe.camunda.io/c2638e36-e194-4c35-a4d9-c'
export ZEEBE_GRPC_ADDRESS='grpcs://c2638e36-e194-4c35-a4d9-d7933878005f.bru-2.zeebe'
export ZEEBE_TOKEN_AUDIENCE='zeebe.camunda.io'
export CAMUNDA_TOKEN_AUDIENCE='zeebe.camunda.io'
export CAMUNDA_CLUSTER_ID='c2638e36-e194-4c35-a4d9-d7933878005f'
export CAMUNDA_CLIENT_ID='H9rdT9~qVTQDKLyF0Pjj7WhK91bKzas'
export CAMUNDA_CLIENT_SECRET='q33jL7WYRa~AMXp0nGjNBP6UmH1d0Seu_HFp0dDHEfXJT-jWSUsI9'
export CAMUNDA_CLUSTER_REGION='bru-2'
export CAMUNDA_CREDENTIALS_SCOPES='Orchestration'
export CAMUNDA_TASKLIST_BASE_URL='https://bru-2.tasklist.camunda.io/c2638e36-e194-4'
```

Download credentials

Save as connector secret

Red Hat Developer Sandbox

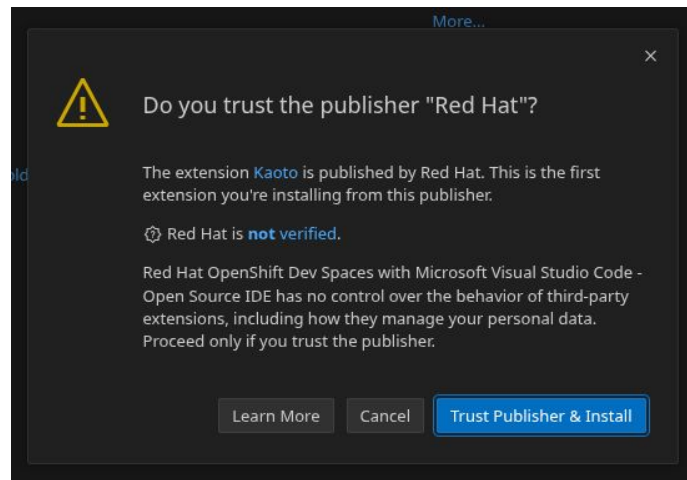
Prerequisite #2

- ▶ <https://developers.redhat.com/developer-sandbox>
- ▶ Deletion after 30 days, possible to start fresh sandbox again

After registration, open the following URL. It will create a new workspace in OpenShift DevSpaces (browser-based IDE)

<https://workspaces.openshift.com/#https://github.com/ibek/pv207-camel-lab>

- 1) if/when DevSpaces prompts for access to github you may decline
- 2) A popup will show up automatically - click on Trust & Install Kaoto extension



EXPLORER

(WORKSPACE)

- pv207-camel-lab
 - .vscode
 - customer-request-triage
 - application-dev.properties
 - BusinessBpmnError.java
 - classify.camel.yaml
 - customer-request-triage.bpmn
 - index.html
 - start-process.camel.yaml
 - triage-classification.schema.json
 - workers.camel.yaml
 - .devfile.yaml
 - .gitignore
 - README.md

! start-process.camel.yaml

pv207-camel-lab > customer-request-triage > ! start-process.camel.yaml

Design Beans Rest DataMapper About

Routes 2/2 + New

Camel Main 4.14.2.redhat-00019

Start process

```
graph LR; direct --> marshal --> setBody --> StartCamunda[Start Camunda pr...] --> log --> AddStep[Add step]
```

Get UI config

```
graph LR; direct --> BuildUI[Build UI config res...] --> AddStep[Add step]
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
pv207-camel-lab (main) $ camel --version
2026-04-09 09:22:30.234 INFO 3604 --- [
2026-04-09 09:22:34.405 INFO 3604 --- [
4.18.1
pv207-camel-lab (main) $
```

Set Camunda client credentials in application.properties

```
camunda.cluster-id=485c3efc-cd09-4c7e-aca5-9a24fa701d45  
camunda.region=bru-2  
camel.component.camunda.clusterId=485c3efc-cd09-4c7e-aca5-9a24fa701d45  
camel.component.camunda.region=bru-2  
camel.component.camunda.clientId=EOJ-YU1S.q_w7OCpZGJ3LOSydCeet7IV  
camel.component.camunda.clientSecret=0Iv10acp2iM4WDpOPdWVgF9Gx~AbJyz-hCK8o4azaabZAs  
SHU1rpBPJ3RkDV.mmE
```

Developer Sandbox

application-dev.properties M Settings ! start-process.camel.yaml M ! start-process.camel.yaml X @ README.md

pv207-camel-lab > customer-request-triage > ! start-process.camel.yaml

Design Beans Rest DataMapper About

Routes 2/2 + New Camel Main 4.14.2.redhat-00019

1. Run: Folder

Start process

direct → marshal → setBody → Start Camunda pr... → log → Add step

Get UI config

direct → Build UI config res... → Add step

CTRL+C to Stop

PROBLEMS **OUTPUT** **DEBUG CONSOLE** **TERMINAL**

```
2026-04-09 11:46:29.647 INFO 5672 --- [
2026-04-09 11:46:29.647 INFO 5672 --- [
0ms init:0ms start:36698ms boot:56374ms)
2026-04-09 11:46:29.649 INFO 5672 --- [
2026-04-09 11:46:29.649 INFO 5672 --- [
2026-04-09 11:46:29.649 INFO 5672 --- [
n/json)
2026-04-09 11:46:29.650 INFO 5672 --- [
/json produce:application/json)
2026-04-09 11:46:29.650 INFO 5672 --- [
application/json)
2026-04-09 11:46:29.650 INFO 5672 --- [
application/json)
2026-04-09 11:46:29.650 INFO 5672 --- [
text/plain,application/json)
2026-04-09 11:46:29.650 INFO 5672 --- [
text/plain,application/json)
2026-04-09 11:46:29.650 INFO 5672 --- [
text/plain,application/octet-stream)
2026-04-09 11:46:29.650 INFO 5672 --- [
multipart/form-data)

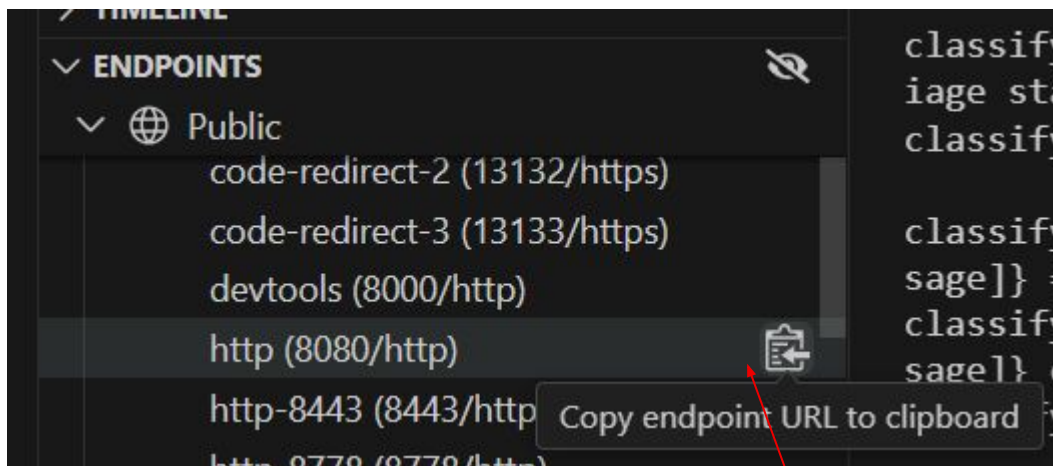
main] e.camel.impl.engine.AbstractCamelContext : Started get-ui-config (rest://get:/api:/config)
main] e.camel.impl.engine.AbstractCamelContext : Apache Camel 4.18.0 (BusinessBpmnError) started in 36698ms (build:
main] nt.platform.http.main.MainHttpServerUtil : HTTP endpoints summary
main] nt.platform.http.main.MainHttpServerUtil : http://0.0.0.0:8080/
main] nt.platform.http.main.MainHttpServerUtil : http://0.0.0.0:8080/api/config (GET) (produce:applicatio
main] nt.platform.http.main.MainHttpServerUtil : http://0.0.0.0:8080/api/start (POST) (accept:application
main] nt.platform.http.main.MainHttpServerUtil : HTTP Management endpoints summary
main] nt.platform.http.main.MainHttpServerUtil : http://0.0.0.0:37545/observe/health (GET) (produce:
main] nt.platform.http.main.MainHttpServerUtil : http://0.0.0.0:37545/observe/info (GET) (produce:
main] nt.platform.http.main.MainHttpServerUtil : http://0.0.0.0:37545/observe/jolokia (GET,POST) (produce:
main] nt.platform.http.main.MainHttpServerUtil : http://0.0.0.0:37545/q/dev (GET) (produce:
main] nt.platform.http.main.MainHttpServerUtil : http://0.0.0.0:37545/q/download (GET) (produc
main] nt.platform.http.main.MainHttpServerUtil : http://0.0.0.0:37545/q/upload (PUT,DELETE) (accept
```

bash
bash
kaoto: Run...

Process http is now listening on port 8080. Open it?

Source: Eclipse Che Port

Open In New Tab



Open in another tab

Customer Request Triage

Apache Camel + Camunda 8 SaaS Demo

Start Process

Customer Tier

Standard

Customer Message

Please cancel order #ORD-100

Start Process

Process Instance

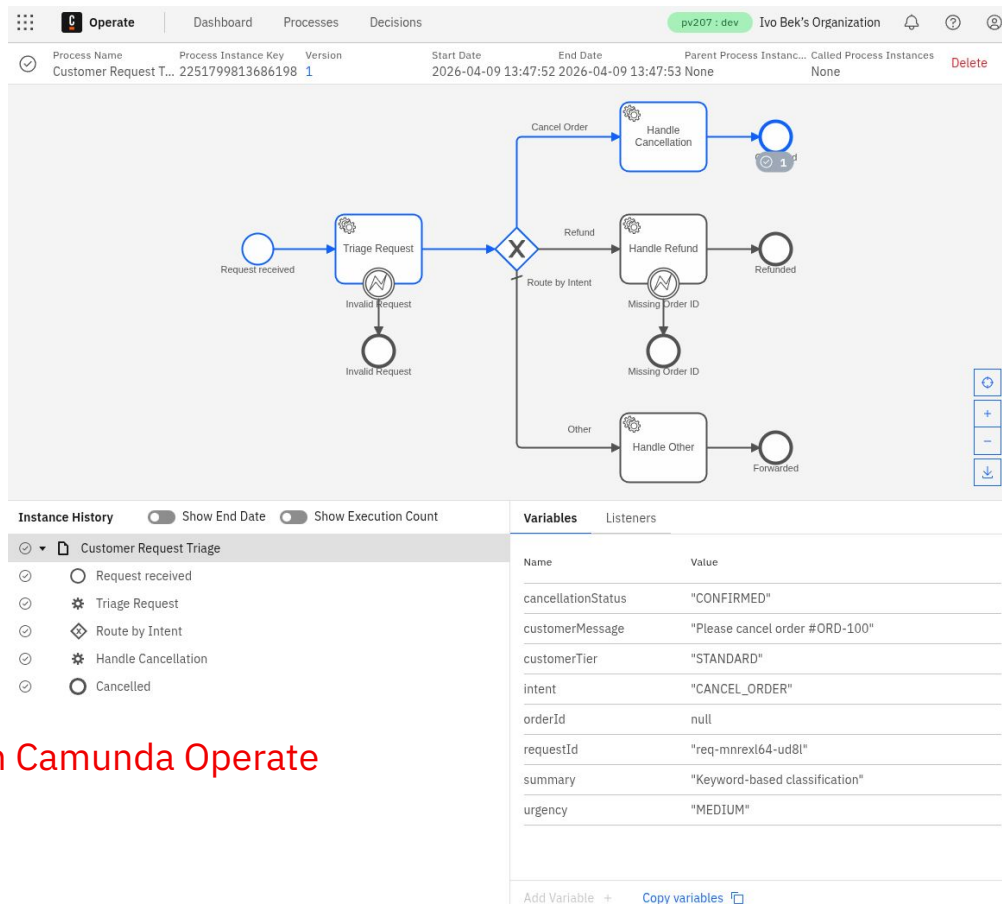
Key: 2251799813686198

Operate: <https://bru-2.operate.camunda.io/c2638e36-e194-4c35-a4d9-d7933878005f/operate/processes/2251799813686198>

1. Enter message from README.md

2. Start Process

3. Open Camunda Operate



(optional) Local setup of Camel development environment



Visual Studio Code



[Kaoto designer for Apache Camel](#)

[Camel JBang \(CLI\) documentation](#)



```
# Linux/Mac - Install JBang

curl -Ls https://sh.jbang.dev | bash -s - app setup

# Windows - Install JBang

iex "& { $(iwr https://ps.jbang.dev) } app setup"

# Install Camel CLI

jbang app install camel@apache/camel

# Check

camel version

# Output: Camel JBang version: 4.18.Z
```

Resources

PV207-camel-lab git repository: <https://github.com/ibek/pv207-camel-lab>

- ▶ [Camel Components](#)
 - [Camel Camunda component](#)
 - [Camel File component](#)
 - [Camel HTTP component \(producer\)](#)
 - [Camel Platform HTTP component \(consumer/server\) - example](#)
 - [Camel OpenAI component](#)
- ▶ [Camel JBang examples](#)
- ▶ [Camel OpenAI Patterns](#)

Thank you!



linkedin.com/company/red-hat



youtube.com/user/RedHatVideos



facebook.com/redhatinc



twitter.com/RedHat



Red Hat

